LAB 1 (25%)
Energy Usage Tracker

LO1	Demonstrate critical judgment in decomposing large tasks into collections of small objects and functions.
LO2	Design scalable object-oriented software with an appreciation of a larger environment encompassing; code recycling, maintenance, expansion and issues of robustness.
LO6	Design & write object orientated code against a design brief.

Total marks: 100

Part A: Overview

In this coursework assessment, you are to create a **C++ program** that simulates tracking energy usage of **exactly 3 household devices from your own daily routine** (e.g., laptop, smart watch, microwave oven). To ensure originality, you must include **personalized elements** such as your own devices, your student ID in the program output, and a custom sorting rule.

This coursework assessment is made available to the students on Monday 12th of October 2026 and is due back uploaded on Moodle by **5pm, Monday 16th November 2026**. Please be sure to submit your programs on Moodle by the due date. Late submissions will incur a penalty based on the *quality manual*. This assessed lab is worth **25%** of your final grade.

*Plagiarism statement: The codes you submit are to be developed by the student alone. Any codes which are **significantly** similar to each other, or to code found on the internet will be penalized as plagiarism and points will be deducted accordingly.*

Part B: Marking Rubric

LO	Criteria	Poor (0 – 25%)	Fair (26 – 50%)	Good (51 – 75%)	Excellent (76 – 100%)
LO1 – Decomposition (8%)	**Problem Decomposition (4%)**	Unclear, no logical breakdown, scattered approach.	Basic attempt; partial structure but gaps remain.	Clear decomposition into manageable parts; mostly logical.	Thorough, well-structured breakdown; logical flow, complete and easy to follow.
	Documentation (4%)	Disorganized, unclear, no headings	Basic structure; some headings.	Clear structure, readable layout.	Professional documentation; well-structured with effective formatting.
LO2 – OOP Design (9%)	**Class Design (3%)**	Weak or incorrect design; lacks classes or structure, no meaningful comments.	Basic design with limited class use; some structure but not robust, minimal comments.	Clear class design, mostly scalable and reusable; minor issues, comments explain most code sections.	Strong, professional design; well-structured, scalable, and robust with effective reuse, clear, consistent code comments..
	Scalability & Robustness (3%)	Fragile design; fails under complexity or change.	Some scalability but limited robustness; basic error handling.	Generally scalable and robust; handles most cases well.	Highly scalable and robust; resilient, efficient, and adaptable.
	Code Recycling & Expansion (3%)	No reuse; repetitive or rigid code.	Limited reuse; some duplication remains.	Reusable components; expansion possible with minor effort.	Well-designed for reuse and expansion; modular and efficient.
LO6 – Implementation (8%)	**Fulfillment of Brief (4%)**	Incomplete or incorrect implementation; fails to meet brief; missing student-specific elements.	Basic implementation; meets parts of the brief but gaps remain; student-specific elements only partially applied.	Functional implementation; meets most requirements includes student-specific elements with minor issues.	Complete, polished implementation; fully meets brief; all student-specific elements (own devices, student ID in output, custom sorting rule) correctly integrated.
	Testing & Evaluation (4%)	Code undeveloped or fails to compile; no testing or evaluation.	Code compiles but runs with errors; minimal testing; output inconsistent.	Code compiles and runs with minor issues; output mostly correct; some testing and evaluation present.	Code compiles and runs smoothly without errors; output consistently accurate; thorough testing and evaluation documented.

Note: Each criterion is marked using the bands above. **LO weights** are percentages of the module **(total 25%).**

*Example: 80% on a criterion worth 4% gives **3.2%***

Part C: Tasks

I. Coursework tasks

1. Accept input for each device:
   - **Device name**
   - **Power rating** (W), and
   - **Usage hours** per day.

2. Use a `Device` class to group attributes and operations.

3. Store and manage these devices using **classes** and **pointers**.

4. Use **functions** to modularize tasks (e.g., input, display, sorting).

5. Demonstrate correct use of **tokens, identifiers,** and **data types**.

6. Implement a **custom sorting rule**:
   - Define your own criterion (e.g., alphabetical order, usage hours, or "favourite device").
   - Explain your chosen rule in the short report.

7. Write a short **report (maximum 5 pages including screenshots)** covering:
   - **Design plan:** decomposition into functions and classes.
   - **OOP principles** used:
     - Modularity (functions with clear responsibilities).
     - Scalability (design extendable to more devices or attributes).
     - Robustness (handling invalid input or errors gracefully).
     - Basic class usage (`Device` class for attributes or operations).
   - **Justification:** explain your custom sorting rule.
   - **Presentation:** report is clear, structured, and professional.

8. Personalization (mandatory): To ensure originality (as stated in **Part A**), your work must include:
   - **Own devices**: Select **3 devices** from your daily routine.
   - **Student ID in output**: Each device must be labeled with your student ID (e.g., "`Laptop20123456`").
   - **Custom sorting rule**: Define and apply your own sorting criterion, explained in your report.

II. Assessment Mapping

LO	Title	Evidence of Achievement	Weight of Module (%)
LO1	Decomposition	• Problem Decomposition: design plan showing decomposition into functions and classes • Documentation: report explains the modular structure clearly	8%
LO2	OOP Design	• Class Design: use of the `Device` class, with clear comments • Scalability & Robustness: demonstrates modularity, scalability and robustness; functions and pointers applied appropriately • Code Recycling & Expansion: reusable components, with expansion possible	9%
LO6	Implementation	• Fulfilment of Brief: meets the brief (own devices, student ID in output, custom sorting rule) • Testing & Evaluation: compiles and runs smoothly; tested and evaluated	8%
		Total	25%

Part D: Submission

Submit a **single ZIP file** via Moodle by **5 pm** on **16th of November 2026.** The **ZIP file** should be named as '***StudentID***_*ModuleCode_AcademicYear_AL1' (e.g.,* ***20123456_EEEE2065_2627_AL1****).* The ZIP file should contain the following:

1. A `.cpp` file with all function definitions.
2. A `.h` file with all function declarations.
3. A `main.cpp` file that tests your class functions.
4. A `.cbp` project file that ensures your project opens directly in Code::Blocks for easy testing.
5. A report in `.pdf` file.